

CS2 AI COACHING SYSTEM

analogy-book

Autore: Renan Augusto Macena

Marzo 2026

Il Libro delle Analogie — Ultimate CS2 Coach

Autore: Renan Augusto Macena **Lingua canonica:** Italiano (le traduzioni in inglese e portoghese seguono in [analogy-book-en.md](#) e [analogy-book-pt.md](#)). **Scopo:** Raccogliere in un unico volume tutte le analogie didattiche che compaiono disseminate nelle quattro parti del *Ultimate CS2 Coach*. Le analogie sono il cuore pedagogico del libro — rendono comprensibile a chiunque un sistema che altrimenti richiederebbe un dottorato in ingegneria del machine learning.

Introduzione

Questa è la sezione che scrive Renan. La sua voce, la sua storia, il perché di queste metafore. Non posso sostituirlo qui. Il paragrafo che segue è un segnaposto finché Renan non lo riempie — non è opera mia, e dovrà essere cancellato o riscritto dall'autore umano prima della pubblicazione.

[Placeholder Renan: racconta qui perché hai scelto analogie concrete — la fabbrica, la scuola, la giuria, l'ospedale — invece di equazioni. Racconta la prima volta che un amico ti ha chiesto "ma come funziona davvero?", e hai risposto con una metafora che ha acceso la lampadina nei suoi occhi. Questa è la lezione più importante del libro, e deve essere raccontata con la tua voce, non con la mia.]

Le analogie in questo volume sono **canoniche**: ognuna fa da punto di riferimento per tutte le varianti che appaiono nei libri dei coach. Quando un capitolo dice *"come la giuria di tre giudici"*, l'intero racconto completo sta qui, nella voce §3 di questo volume. I libri possono quindi concentrarsi sulla precisione tecnica — dimensioni, costanti, invarianti — mentre la comprensione profonda resta preservata in questa raccolta.

Questa separazione non è un compromesso — è un guadagno. I libri tecnici del coach sono densi per necessità: devono citare file e righe, descrivere invarianti matematiche, specificare parametri e contratti. Se dentro quella densità si inserissero le metafore a piena lunghezza, il lettore tecnico troverebbe le analogie come un rallentamento; ma le analogie sono indispensabili per chi non vive nel codice. Separandole in questo volume, abbiamo due libri complementari: uno per chi deve *implementare*, uno per chi deve *capire*. I lettori migliori useranno entrambi.

Come leggere questo libro

1. **Apri il capitolo del libro-coach che stai studiando** (Parte 1A, 1B, 2 o 3). Quando incontri una riga `> **Analogia:** [titolo – vedi Libro delle Analogie §X.Y]`, torna qui.

2. **Leggi l'analogia canonica** in questo volume (sezione corrispondente).
3. **Ritorna al libro** e continua la lettura tecnica: a quel punto la metafora è fissata e il resto del capitolo può parlare di `METADATA_DIM=25` , `GLOBAL_SEED=42` e `input_dim=153` senza perdere il filo narrativo.
4. **Cerca i confronti** (sezione "Confronta con" in fondo a ogni voce): molte analogie si illuminano a vicenda.

Il volume è organizzato in **dieci sezioni tematiche** che corrispondono ai grandi blocchi dell'architettura del coach:

- **§1 — Nucleo Neurale** (7 voci): i modelli che fanno il pensiero centrale — MoE, Superposition, ResNet, Value Critic, GhostEngine.
- **§2 — Memoria e Tempo** (3 voci): come il coach ricorda, cosa dimentica, e la scala temporale delle sue osservazioni.
- **§3 — Percezione e Dati** (2 voci): i cinque sensi del coach e il contratto canonico a 25 dimensioni che li unifica.
- **§4 — Strategia e Coaching** (5 voci): il piano di battaglia, la catena di fallback dei servizi, l'attribuzione dei ruoli narrativi.
- **§5 — Conoscenza e Recupero** (3 voci): come il coach sa cosa sa — biblioteca FAISS, diario COPER, grafo dei fatti.
- **§6 — Addestramento e Orchestrazione** (4 voci): la scuola, l'orchestra, l'esame, il talent scout — come si cresce dal nulla a master chef.
- **§7 — Motori di Analisi** (3 voci): i detective specializzati che vedono cose che gli altri motori non vedono.
- **§8 — Dati e Ingestione** (3 voci): l'ufficio postale, il doganiere, il cronista — come i demo arrivano e vengono capiti.
- **§9 — Database e Storage** (2 voci): l'archivio a tre livelli, la cassaforte del backup.
- **§10 — Visione d'Insieme** (3 voci): il robot che guarda video, la città del coach, il cruscotto a cinque strati.

Legenda — dove nasce ogni analogia

Questa tabella mappa le **35 voci canoniche** alle loro posizioni originali nei quattro libri. Utile quando un lettore vuole tornare al contesto tecnico dove la metafora è stata introdotta per la prima volta.

Voce	Titolo	Fonte (Libro : riga)
§1.1	La Fabbrica dei 6 Reparti	1A:84
§1.2	La Giuria dei 3 Giudici	1A:329, 1B:230
§1.3	Il Mixer a 256 Canali	1A:854, 1B:236
§1.4	L'Edificio a 7 Piani	1B:53, 1B:147

Voce	Titolo	Fonte (Libro : riga)
§1.5	I Tre Occhiali dell'Allenatore	1B:119
§1.6	L'Ologramma del Miglior Te	1B:514, 3:1605
§1.7	La Pagella Mentale	1A:1134, 1A:1250
§2.1	L'Album di Hopfield	1B:172, 1B:164, 1B:184
§2.2	Il Mentore Lento (EMA)	1A:399, 1A:872
§2.3	Le Tre Lenti del Chronovisor	1B:449, 1B:459
§3.1	I 5 Sensi del Coach	1B:620, 1A:267, 1B:932
§3.2	Le 25 Domande del Sensore	1A:937, 3:1195
§4.1	L'Ospedale e la Reception	2:50, 2:89
§4.2	Il Detective dei 5 Errori	1B:282, 1B:307
§4.3	Il Commentatore del Valore	1B:268
§4.4	L'Albero di Scacchi CS2	2:886, 2:921
§4.5	Il Poker-Face dell'Inganno	2:964
§5.1	La Biblioteca FAISS	1B:1032, 2:589
§5.2	Il Libro di Testo + Diario	2:573, 2:620, 2:700
§5.3	Il Grafo dei Fatti	2:714
§6.1	La Scuola del Coach	1A:878, 3:399, 3:431
§6.2	L'Orchestra del Training	2:2228, 2:2462, 2:2681
§6.3	L'Esame con Risposte sul Retro	1A:599, 3:556, 3:574, 3:591
§6.4	Il Talent Scout dei Ruoli	2:808, 2:812, 2:1722
§7.1	La Squadra dei Detective	2:154, 2:776
§7.2	L'Anello dell'Umore	2:990, 2:1370
§7.3	L'Istruttore di Guida	2:1043, 2:1136
§8.1	L'Ufficio Postale della Lettera	3:1107, 3:2626
§8.2	Il Doganiere Demo	1B:702, 2:1388
§8.3	Il Cronista Sportivo HLTV	1B:677, 1B:779, 3:2667
§9.1	L'Archivio a 3 Livelli	3:100, 3:1369, 2:2047, 2:2083
§9.2	La Cassaforte del Backup	2:2134, 3:1885
§10.1	Il Robot che Guarda Video	1A:62
§10.2	La Città del Coach	3:618, 3:841

Voce	Titolo	Fonte (Libro : riga)
§10.3	Il Cruscotto a Cinque Strati	3:944, 3:2905, 3:2984

§1 — Nucleo Neurale

§1.1 La Fabbrica dei 6 Reparti

Pensa al sistema come a una **grande fabbrica con sei reparti collegati**. Il primo reparto (Ingestione) è la **sala posta**: riceve le registrazioni grezze delle partite — i file `.dem` — e le ordina per mappa, squadra e data. Il secondo (Elaborazione) è l'**officina**: prende le registrazioni e misura ogni tick — posizione, arma, salute — trasformando bit grezzi in un vettore canonico a 25 dimensioni. Il terzo (Formazione) è la **scuola**: istruisce il cervello dell'IA mostrandogli migliaia di esempi, progredendo dai pistoli ai full buy. Il quarto (Conoscenza) è la **biblioteca**: memorizza suggerimenti, esperienze passate e riferimenti professionisti in modo che l'allenatore possa consultarli. Il quinto (Inferenza) è il **cervello**: combina ciò che l'IA ha imparato con ciò che la biblioteca conosce per creare consigli. Il sesto (Analisi) è la **squadra investigativa**: conduce indagini speciali — *"questo giocatore è in difficoltà?"*, *"era una buona posizione?"* — usando undici motori analitici. Tutti e sei lavorano in parallelo: quando apri il desktop, un daemon Scanner scansiona le cartelle, un Digester analizza i demo, un Teacher aggiorna i modelli e un Pulse tiene sotto controllo la salute complessiva. I sei reparti di questa fabbrica corrispondono esattamente alle sottodirectory di `backend/`: `ingestion/`, `processing/`, `nn/`, `knowledge/`, `services/`, `analysis/`. La fabbrica non è un'azienda sprecona — ogni reparto ha uno scopo preciso, e nessun reparto fa il lavoro di un altro. Un demo che arriva alla sala posta non viene mai processato in officina finché non è stato registrato; un modello non viene mai messo in produzione finché la scuola non lo ha promosso; un consiglio non viene mai dato senza che la biblioteca e il cervello abbiano prima concordato. L'organizzazione garantisce che un guasto in un reparto — un demo corrotto, un training crash, un lookup fallito — non paralizza gli altri.

Confronta con: §6.1 (La Scuola del Coach — il terzo reparto visto da dentro), §9.1 (L'Archivio — come il quarto reparto conserva la memoria), §10.2 (La Città — la stessa fabbrica vista come infrastruttura urbana).

§1.2 La Giuria dei 3 Giudici

Alcuni modelli del coach funzionano come una **giuria di tre giudici a un talent show**. L'LSTM legge la sequenza di gioco come se leggesse una storia — capisce cosa è successo passo dopo passo, ricordando i momenti importanti — e produce un riepilogo in 128 numeri, la sua "opinione". A quel punto, tre esperti esaminano il riepilogo e assegnano ciascuno il proprio punteggio. Ma non tutti i giudici sono ugualmente competenti in ogni situazione: un esperto di tiro giudica meglio i duelli, un esperto di economia giudica meglio gli eco-round, un esperto di posizionamento giudica meglio le setup difensive.

Per questo motivo una **rete di gating** — come un moderatore — decide quanto fidarsi di ciascun giudice in questo specifico momento: *"Per questo tick, il Giudice 1 pesa 60%, il Giudice 2 pesa 30%, il Giudice 3 pesa 10%"*. Il punteggio finale è la media pesata delle tre opinioni. Nel RAP Coach la stessa metafora si estende a **quattro generali** riuniti nella sala strategica (`SuperpositionLayer` + `MoE` , top-k=2), dove un gate softmax sceglie i due più rilevanti e li lascia discutere tra loro prima di produrre la decisione collettiva. È così che la rete evita di essere un generalista mediocre: si specializza, e la specializzazione si attiva solo quando serve. Il meccanismo Mixture-of-Experts è uno dei più eleganti nel machine learning moderno — invece di una rete monolitica che cerca di sapere tutto, si addestrano molti piccoli specialisti e si impara contestualmente chi interpellare. Un singolo generale esperto di mid-round non verrebbe mai chiamato durante un clutch 1v1; un singolo esperto di clutch non verrebbe mai chiamato durante una setup di eco. Il gate non è un parametro fisso — è una rete piccola che si allena insieme al resto, imparando a instradare bene i casi.

Confronta con: §1.3 (Il Mixer — come si modula l'intensità di ciascun giudice), §6.2 (L'Orchestra — come i giudici vengono addestrati insieme).

§1.3 Il Mixer a 256 Canali

Immagina un **mixer audio con 256 canali e migliaia di potenziometri** che si muovono da soli in base al contesto: quando suoni un passaggio jazz, solo i canali giusti si alzano; quando suoni rock, altri canali prendono il comando. Il `SuperpositionLayer` funziona così. Ogni neurone è un canale; per ogni tick di gioco, il contesto — ruolo del giocatore, mappa, fase del round — determina automaticamente quali neuroni devono accendersi e quali restare silenziosi. Un termine di **sparsità L1** nel training spinge il modello a usare il minor numero possibile di neuroni per arrivare alla risposta giusta — come un fonico bravo che ottiene un suono pulito con pochi canali attivi anziché schiacciare tutto al massimo. Il risultato è un modello che **riutilizza lo stesso cervello** per dieci ruoli e sette mappe senza cadere nella trappola di essere "bravo a tutto, esperto in nulla". Ogni neurone è come un dimmer, e il contesto decide la luminosità: quando un AWP gioca su Nuke nel side T, si attivano esattamente i neuroni che servono per quel problema specifico. La bellezza del meccanismo è che la sparsità non è imposta rigidamente — emerge naturalmente dall'ottimizzazione. Il training mostra al modello che usare 80 neuroni su 256 per questo ruolo è "premiato" (loss più bassa) rispetto a usarli tutti; e il modello, incentivato, impara automaticamente a specializzarsi. La sparsità L1 è più dolce della L0 (che conterebbe i neuroni attivi in modo discreto e sarebbe non differenziabile): penalizza la somma dei valori assoluti dei pesi, spingendo verso lo zero quelli che non servono ma senza obbligarli a essere esattamente zero. È il compromesso perfetto tra pulizia e addestrabilità.

Confronta con: §1.2 (La Giuria — i giudici che il mixer "illumina"), §6.3 (L'Esame — la sparsità L1 vista come criterio d'esame).

§1.4 L'Edificio a 7 Piani

Il RAP Coach — il modello neurale principale del sistema — è un **edificio di sette piani** dove ogni piano ha una funzione specializzata. Il primo piano è la **Percezione**: tre flussi convoluzionali processano la vista del giocatore, la minimappa e il delta di movimento (vedi §1.5). Il secondo è la **Memoria**: LSTM + Hopfield tengono traccia del contesto recente (vedi §2.1). Il terzo è la **Strategia**: SuperpositionLayer e Mixture-of-Experts scelgono il tipo di consiglio (vedi §1.3). Il quarto è la **Pedagogia**: il Value Critic stima "quanto è buona questa situazione" e un Skill Model adatta il consiglio al livello del giocatore. Il quinto è la **Posizione**: prevede dove il giocatore dovrebbe muoversi (vedi §1.6). Il sesto è l'**Attribuzione Causale**: spiega *perché* qualcosa è andato male (vedi §4.2). Il settimo è l'**Output**: assembla tutti i risultati in un singolo dizionario pronto per il livello applicativo. Tra i piani esistono **ascensori identità** (skip connections in stile ResNet) che permettono all'informazione di saltare i piani e arrivare subito in cima quando serve — un trucco essenziale per addestrare reti profonde senza che il gradiente si dissolva. Senza gli ascensori, alle profondità del sesto piano il segnale sarebbe ormai troppo sbiadito per essere utile. Ogni piano ha ingressi e uscite ben definiti — un contratto che permette di sostituire un piano con una variante migliore senza rompere il resto dell'edificio. È stato così che il progetto ha potuto evolvere: quando il piano di Percezione è stato aggiornato (3 flussi invece di 2), gli altri piani non hanno dovuto essere riscritti; quando il piano di Memoria ha ricevuto l'aggiunta di Hopfield accanto all'LSTM, il piano di Strategia non ha notato la differenza. Questa modularità è il segreto di come il RAP Coach ha potuto raggiungere una complessità che sarebbe stata ingestibile in un'architettura monolitica.

Confronta con: §1.5 (i Tre Occhiali — dettaglio del piano 1), §2.1 (L'Album — dettaglio del piano 2), §4.2 (Il Detective — dettaglio del piano 6).

§1.5 I Tre Occhiali dell'Allenatore

Il livello di Percezione del RAP Coach è come **tre paia di occhiali** indossati simultaneamente. Il **primo paio** (flusso ventrale) mostra cosa vede il giocatore — la sua prospettiva in prima persona — processato da una ResNet leggera che estrae 64 caratteristiche dall'immagine. Il **secondo paio** (flusso dorsale) mostra la **minimappa aerea** — dove si trova chiunque — processato da una rete più semplice in 32 caratteristiche. Il **terzo paio** mostra il **delta di movimento** — chi si sta muovendo e con quale velocità — in altre 32 caratteristiche. Tutte e tre le viste vengono concatenate in un unico vettore di 128 numeri: *"Ecco tutto ciò che riesco a vedere in questo istante"*. La separazione è ispirata al cervello umano: il flusso ventrale riconosce *cosa* sono le cose, mentre il dorsale traccia *dove* si trovano. Cruciale: tutti gli input sono costruiti dal **TensorFactory** rispettando il principio **NO-WALLHACK**, cioè il coach vede solo quello che vede il giocatore. Non esiste telecamera esterna che tradisce le informazioni nascoste dagli ostacoli. Se un giocatore non può vedere il nemico dietro una curva, il coach non può penalizzarlo per non averlo anticipato. È un esame di guida in cui l'istruttore giudica solo ciò che l'allievo ha ragionevolmente potuto vedere. Il NO-WALLHACK non è solo una regola etica — è una regola di *equità pedagogica*. Un coach che sapesse dove si trovano i nemici nascosti insegnerebbe al giocatore strategie impossibili da replicare nella realtà, dove i nemici restano nascosti. Il coach deve vivere nella stessa nebbia di guerra del giocatore — ed è da quella nebbia condivisa che emerge la fiducia: *"il mio coach mi consiglia una giocata possibile, non una giocata che richiede poteri che non ho"*.

Confronta con: §3.1 (I 5 Sensi — come i dati arrivano ai tre occhiali), §10.1 (Il Robot che Guarda Video — la percezione end-to-end).

§1.6 L'Ologramma del Miglior Te

Il **GhostEngine** produce un **ologramma trasparente** sulla mappa tattica che mostra *dove avresti dovuto trovarti*. In ogni tick, chiede al RAP Coach: *"Data questa situazione esatta, dove DOVREBBE trovarsi il giocatore?"*. La risposta è un piccolo delta posizionale — per esempio *"cinque unità a destra, tre in avanti"* — scalato di $\times 500$ in coordinate di mappa CS2. Il risultato è un *"tu ideale"* fantasma disegnato sulla minimappa, che corre accanto al giocatore reale. Se il fantasma è lontano, sai che eri in una brutta posizione; se ti è vicino, il tuo posizionamento era corretto. L'inferenza avviene in tempo reale grazie a un **FrameBuffer** circolare che tiene gli ultimi 32 tick — un "registratore a nastro circolare" — e il motore di inferenza mantiene lo stato LSTM tra tick consecutivi (il famoso **hidden_state** del settimo output del RAP Coach), così non "dimentica" cosa è successo cinque secondi fa quando ti valuta ora. Il ghost non è un giudizio definitivo sul tuo gioco: è un riferimento visivo, un "miglior te" che appare dove il coach pensa tu avresti dovuto essere, e scompare quando coincidi con lui. La bellezza del ghost è che non è prescrittivo in modo autoritario — è suggestivo in modo gentile. Non ti dice "hai sbagliato"; mostra *dove sarebbe meglio essere*, e lascia a te la scelta di seguirlo. Dopo un po' di ore di pratica con il ghost visibile, i giocatori riportano di sentire la sua presenza anche quando è spento — hanno interiorizzato il pattern, e non hanno più bisogno dell'ologramma perché adesso pensano già come lui. Il ghost è quindi un insegnante transitorio: ti aiuta a crescere e poi diventa superfluo.

Confronta con: §3.2 (Le 25 Domande — l'input che alimenta l'ologramma), §4.3 (Il Commentatore del Valore — giudizio sul "quanto è buona" la posizione).

§1.7 La Pagella Mentale

Mentre il coach si allena, un **Osservatorio di Introspezione** tiene una pagella della sua salute mentale. Non guarda solo la perdita totale — sarebbe come giudicare uno studente dalla sola media finale. Misura invece **cinque segnali** che compongono un *Conviction Index* da 0 a 1: l'entropia delle credenze (sta ancora esitando?), la specializzazione delle gate (ogni esperto sta imparando il proprio ruolo?), il focus concettuale (i 16 concetti VL-JEPA si stanno differenziando?), l'accuratezza del valore (le stime del Value Critic migliorano?) e la stabilità del ruolo (le predizioni oscillano o sono coerenti?). Questi cinque segnali definiscono quattro stati di maturità: **DOUBT** (< 0.30 — il modello si sta ancora chiedendo chi è), ****LEARNING**** ($0.30-0.60$ — sta studiando), ****CONVICTION**** (> 0.60 stabile per 10+ epoche — ha formato una visione), **MATURE** (> 0.75 stabile per 20+ epoche — può andare in produzione). Una caduta brusca $> 20\%$ attiva lo stato **CRISIS**, perché una perdita improvvisa di sicurezza è più sospetta di un errore singolo. Il medico non giudica mai un paziente dal battito cardiaco in un istante: guarda la tendenza. Il *Conviction Index* fa lo stesso per il cervello dell'IA, e alimenta TensorBoard per dashboard in tempo reale. L'Osservatorio non è solo strumentazione — è filosofia: un modello che sta imparando male deve essere fermato prima che diventi un modello che ha imparato male. Alla prima manifestazione di

CRISIS, il training viene pausato e le ultime epoche vengono esaminate: è un bug nel dataset? una sparsità L1 troppo aggressiva? un drift nelle statistiche del preprocess? Questa disciplina di auto-diagnosi è ciò che distingue un sistema ML ingegnerizzato da un esperimento che "speriamo funzioni".

Confronta con: §6.3 (L'Esame — le singole perdite misurate dall'Osservatorio), §6.2 (L'Orchestra — i callback che scrivono sulla pagella).

§2 — Memoria e Tempo

§2.1 L'Album di Hopfield

Il livello di Memoria del RAP Coach è un **album fotografico di giocate famose**. La rete di Hopfield è la parte associativa: ha memorizzato durante l'addestramento alcuni pattern ricorrenti — *"fast push attraverso il mid di Mirage"*, *"retake B di Inferno con stack di 4 utilità"*, *"clutch 1v2 con pistol a round 12"* — e, data una situazione nuova, recupera il pattern più simile come quando ti viene in mente una giocata di un Major vedendo un frammento di quella attuale. Accanto all'album c'è un **cervello vivo LTC** (Liquid Time-Constant network): non è una foto, è un organismo che si adatta al ritmo del gioco — quando la partita accelera, il cervello segue; quando rallenta, si rilassa. Un dettaglio cruciale: **l'Hopfield è bypassato per i primi due forward pass di training** (invariante `NN-MEM-01` in `rap_coach/memory.py:74-78`), perché un albumista che non ha ancora memorizzato nulla non può scegliere foto significative — sceglierebbe a caso, rovinando i primi gradienti. Durante quei passaggi solo l'LSTM guida. A 153 dimensioni in entrata (128 percezione + 25 metadati) esce uno stato nascosto a 256 dimensioni più un vettore di 64 credenze — *"cosa penso stia accadendo"* — che alimenta il resto dell'edificio a 7 piani. Se l'hardware è modesto esiste il `RAPMemoryLite`, un generatore di backup con carburante più semplice che funziona quando l'edificio principale non riesce a caricare l'Hopfield completo. La sinergia LSTM + Hopfield è preziosa: l'LSTM gestisce la cronologia *temporale* (cosa è successo tick per tick negli ultimi secondi) e l'Hopfield gestisce la memoria *associativa* (quale pattern simile è stato visto in altre partite). Insieme coprono due dimensioni complementari: "cosa è successo a me di recente" e "cosa succede quando il gioco si trova in questa situazione generica". Il coach attinge da entrambe per costruire una comprensione che è simultaneamente personale e universale.

Confronta con: §1.4 (L'Edificio — il piano dove vive l'album), §2.3 (Le Tre Lenti — come si sceglie *quando* ricordare), §5.2 (Il Libro di Testo + Diario — memoria esterna che complementa quella interna).

§2.2 Il Mentore Lento (EMA)

Quando JEPA si addestra, affianca a ogni encoder un **"fratello maggiore" EMA** (Exponential Moving Average) che assorbe le conoscenze del fratello più giovane *molto lentamente*. Il fratello minore — l'online encoder — fa gli esperimenti: prova, sbaglia, corregge, cambia pesi ogni passo. Il fratello maggiore — il target encoder — è il mentore zen: ogni passo, il 99.6% resta com'era, solo lo 0.4% si aggiorna verso ciò che il minore ha scoperto. Questo "obiettivo lento" impedisce al sistema di **collassare** in una soluzione

banale ("tutto uguale, tutto ha embedding zero, soluzione accettata"). In JEPA il momentum parte da **0.996** e segue una **schedulazione coseno** verso **1.0** durante l'addestramento (`backend/nn/jepa_train.py:353`): all'inizio il target encoder traccia velocemente l'online encoder, ma man mano che la rete matura, il target si fossilizza e diventa un riferimento stabile — come un mentore che all'inizio è curioso di ogni idea dell'allievo e poi, quando ha visto abbastanza, diventa fermo. Il RAP Coach usa una variante separata con decay 0.999 (`backend/nn/ema.py:39`). Regola d'oro implementativa: `apply_shadow()` DEVE fare `.clone()` dei pesi shadow (invariante `NN-16` in `ema.py:79`) — altrimenti l'aliasing provoca corruzione quando si applicano/ripristinano i pesi del mentore durante l'inferenza. La metafora del fratello maggiore spiega anche un'altra regola tecnica: durante l'update EMA, il target encoder DEVE avere `requires_grad=False` (invariante `NN-JM-04` in `jepa_model.py:323-331`). Se fosse ancora addestrabile, l'ottimizzatore cercherebbe di cambiare anche lui, e il mentore lento diventerebbe un mentore confuso. La separazione dei ruoli è essenziale: l'online encoder impara dai gradienti, il target encoder impara solo dall'EMA. Violare questa regola produce un training che sembra normale ma collassa silenziosamente dopo qualche centinaio di epoche.

Confronta con: §6.3 (L'Esame — come si misura se il mentore sta imparando bene), §2.3 (Le Lenti — l'altra dimensione temporale del training).

§2.3 Le Tre Lenti del Chronovisor

Il `ChronovisorScanner` cerca i momenti importanti di una partita usando **tre lenti di ingrandimento** diverse. La prima lente è il *zoom micro* (scala tick-by-tick): vede il singolo duello, il singolo flash, il singolo salto. La seconda è il *zoom standard* (scala round): vede la traiettoria economica del round, il ritmo degli ingaggi, il posizionamento a ridosso del timer bomba. La terza è il *zoom macro* (scala partita): vede il momentum inter-round, la mappa tattica complessiva, i pattern che si ripetono ogni sei round. Senza le tre scale, il coach rischierebbe di perdersi: troppa micro e non vede la guerra delle economie; troppa macro e non coglie il kill clutch decisivo. È lo stesso principio di Google Maps: se sei a livello strada non capisci la topografia della regione, se sei a livello continente non vedi la strada. Le tre lenti producono tre stream paralleli di highlight che l'inferenza combina. Implementativamente il Chronovisor decide cosa inviare al GhostEngine e cosa salvare nell'Experience Bank (vedi §5.2) per consultazioni future. Un momento importante è tale solo se si qualifica in *almeno una* delle tre scale — un criterio generoso perché la memoria del coach deve essere inclusiva: è meglio ricordare troppo che dimenticare un momento che in retrospettiva si rivelerà una lezione. Le tre lenti hanno anche la funzione di calibrare *il linguaggio* del coach. Un commento sulla scala micro ("la tua mira era di 3 unità a sinistra del target") suona diverso da un commento sulla scala macro ("hai speso troppo presto la granata"). Il coach sceglie il livello di dettaglio in base alla lente attiva: riporta dettagli granulari solo quando servono, e usa linguaggio astratto per insights ad alto livello.

Confronta con: §7.2 (L'Anello dell'Umore — un'altra metrica multi-scala, inter-round), §5.2 (Il Diario — dove vengono salvati i momenti che le lenti trovano).

§3 — Percezione e Dati

§3.1 I 5 Sensi del Coach

Le sorgenti dati esterne sono i **cinque sensi** dell'allenatore IA. La **vista** è il `DemoParser` che legge ogni tick del file `.dem` attraverso `demoparser2` (versione 0.40.2), preservando lo stato completo del gioco — posizioni, salute, equipaggiamento, angoli di vista, 128 volte al secondo, senza decimazione (la decimazione dei tick è un'invariante proibita in CLAUDE.md, perché perdere anche uno solo romperebbe la continuità che la memoria LSTM richiede). L'**udito** è l'`EventRegistry` che riconosce tutti gli eventi di CS2 — kill, flash, utilità, bomb plant, round end — associandoli a semantica tattica. Il **tatto** è il `TradeKillDetector` che percepisce quando un compagno cade subito dopo un altro e il coach valuta "siamo in trade window". L'**olfatto** è lo `SteamLocator`, il segugio che annusa tra le sottocartelle di Steam per trovare automaticamente i demo CS2 senza chiedere aiuto all'utente. Il **sesto senso** — la telepatia del coach — sono i dati HLTV: ricevuti tramite uno scraper dedicato con Playwright e FlareSolverr, permettono al coach di confrontarsi con i giocatori professionisti viventi, aggiornati settimanalmente. Regola sacra di tutto il sistema sensoriale: vale il **principio NO-WALLHACK**. Il coach processa solo ciò che il giocatore poteva vedere o sentire in quel momento. Mai telecamere di sorveglianza, mai informazione privilegiata. L'esame di guida vale per tutti i sensi: si giudica solo ciò che l'allievo poteva ragionevolmente percepire. La coerenza sensoriale è la base della **credibilità del coaching**: se il coach sa qualcosa che il giocatore non sapeva, i suoi consigli diventano irrealizzabili — *"saresti dovuto andare a retake perché lì c'era un solo nemico"* suona utile ma è inutile se il giocatore non aveva modo di sapere che c'era un solo nemico. Il coach che rispetta i cinque sensi produce consigli che il giocatore può davvero seguire: non "hai sbagliato perché non sapevi X", ma "hai sbagliato perché hai interpretato male Y, che era visibile".

Confronta con: §1.5 (I Tre Occhiali — come la vista diventa tensore), §8.3 (Il Cronista HLTV — il sesto senso visto da vicino), §10.2 (La Città — dove i sensi raccolgono dati).

§3.2 Le 25 Domande del Sensore

Ogni tick di gioco viene tradotto in un **questionario standard a 25 domande**, sempre lo stesso, sempre nello stesso ordine. Il vettore ha 25 dimensioni e la costante `METADATA_DIM = 25` in `backend/processing/feature_engineering/vectorizer.py:32` è l'autorità unica: se cambi il numero di domande senza aggiornare `FEATURE_NAMES`, `METADATA_DIM` e `input_dim` di tutti i modelli simultaneamente, l'intero sistema esplode con errori di moltiplicazione matriciale — e a ragione, perché ogni modello addestrato su una versione del questionario non può capire una versione diversa. Le domande sono: *Quanta vita? Quanto armor? Helm? Defuser? Valore equipaggiamento? Accucciato? Scopato? Flashato? Quanti nemici visibili? Posizione x/y/z? Direzione di sguardo (sin/cos dell' yaw + pitch)? Penalità Z per mappe multi-livello? Rating KAST stimato? ID mappa? Fase round? Classe arma? Tempo nel round? Bomba piantata? Compagni vivi? Nemici vivi? Economia di squadra?* — 25 numeri, tutti normalizzati tra -1 e +1 o 0 e 1. È il **contratto canonico**: se hai 128 sensori attaccati al giocatore che

leggono simultaneamente queste 25 grandezze 128 volte al secondo, hai una descrizione completa dello stato di gioco da cui tutto il coaching può essere derivato. Il **FeatureExtractor** è il **traduttore universale** — il dizionario ufficiale — che tutto il sistema consulta: addestramento, inferenza, RAG, COPER. Se uno qualsiasi usasse un dizionario diverso, la comunicazione interna si romperebbe silenziosamente. Il contratto a 25 dimensioni non è stato scelto a caso — è il risultato di iterazioni: si è partiti da un vettore più piccolo (12 dimensioni) che perdeva informazioni chiave; si è passati a uno intermedio (19) che mancava ancora di alcune normalizzazioni; si è arrivati a 25 solo quando ogni singola domanda risultava *necessaria e non ridondante*. Aggiungere una 26ª domanda è stato discusso più volte (per esempio "tempo trascorso dall'ultimo kill"), ma ogni proposta è stata scartata perché derivabile dalle 25 esistenti o perché la derivazione era meglio lasciata ai modelli stessi.

Confronta con: §1.5 (I Tre Occhiali — le domande visive non entrano qui, entrano nei tensori 64×64), §9.1 (L'Archivio — dove i 25 numeri vengono conservati).

§4 — Strategia e Coaching

§4.1 L'Ospedale e la Reception

Il sottosistema dei **Servizi di Coaching** è come la **reception di un ospedale**. Prende i referti del medico specialista (RAP Coach, §1.4), le note dell'infermiere (motori di analisi, §7.1), i risultati di laboratorio (RAG + COPER, §5) e li trasforma in un rapporto chiaro per il paziente (il giocatore). Ha anche un piano di riserva per ogni scenario: se lo specialista COPER non è disponibile perché i modelli non sono maturi, ti indirizza al medico ibrido (**Hybrid**); se il medico ibrido è assente, ti consegna un opuscolo pre-stampato (**RAG**); e se tutto il resto fallisce, almeno ti fornisce i tuoi parametri vitali di base (**Base**). Il paziente (giocatore) se ne va **SEMPRE** con qualcosa di utile, mai a mani vuote. Questo è il cuore della *catena di fallback a 4 livelli* (COPER → Hybrid → RAG → Base): non c'è mai un "silenzio" del coach. Come scegliere tra pizzeria fine-dining, trattoria, panetteria, pane e acqua: dipende da quanto è matura la cucina oggi. Implementativamente questo si riflette in **CoachingService** che legge lo stato di maturità del coach (CALIBRATING/LEARNING/MATURE) e sceglie quale pipeline attivare. L' **OllamaCoachWriter** (llama3.1:8b locale, opzionale) è la **receptionist poliglotta** che traduce il linguaggio asciutto dei modelli in un italiano motivante e umano. La catena di fallback è una scelta architettuale deliberata: è facile costruire un sistema che funziona benissimo quando tutto è pronto e esplode quando manca qualcosa. È molto più difficile costruire un sistema che degrada con grazia — un consiglio peggiore è sempre meglio di nessun consiglio, soprattutto per un utente che sta imparando. Il coach non è perfetto al primo demo; ma non lascia mai il giocatore senza una parola. Ed è proprio la degradazione graziosa che fa crescere la fiducia del giocatore: anche quando il coach è giovane e poco addestrato, c'è sempre qualcosa di utile da dire.

Confronta con: §5.2 (Il Libro di Testo + Diario — le due fonti di conoscenza che la reception integra), §7.1 (La Squadra dei Detective — i referti che arrivano alla reception).

§4.2 Il Detective dei 5 Errori

L' **AttributionHead** del RAP Coach è il **detective causale**: dopo ogni decisione, non si limita a dire "hai sbagliato", ma risponde a *"perché hai sbagliato"*. Produce un vettore a 5 dimensioni — cinque categorie di colpa — che si legge come una pagella scolastica: **Posizionamento** (eri nel posto giusto?), **Timing** (hai peekato al momento giusto?), **Mira** (il crosshair era dove doveva essere?), **Utility** (granata sprecata o mai usata?), **Economia** (hai comprato nel momento giusto?). Ogni decisione riceve una ripartizione — per esempio 60% timing, 25% mira, 15% posizionamento — e l'interfaccia mostra al giocatore le categorie dominanti con raccomandazioni specifiche. Funziona accanto a un modello di skill latente che descrive il giocatore su cinque materie con Z-score e percentili: *"sei al 72° percentile in aim sui pro, al 34° in economy"*, e la perdita composita dell'addestramento è una **pagella a 4 voci** (MSE per posizione, MSE per valore, Cross-Entropy per strategia, KL per attribuzione) pesate insieme (`rap_coach/trainer.py:27` per i pesi Z-axis). Il detective non punta il dito per colpevolizzare — punta il dito per insegnare: ogni errore è classificato in una materia precisa, così lo studente sa esattamente su cosa esercitarsi nella prossima sessione di DM. La pedagogia del detective è radicale nella sua gentilezza: non accetta la narrativa del "giocatore cattivo" — per il detective non esiste un giocatore cattivo, esiste una distribuzione di errori in 5 materie, e ciascuna materia è migliorabile con esercizio mirato. Questa framing è liberatoria per l'utente: invece di sentirsi dire "hai perso perché sei scarso", sente "hai perso il 40% dei duelli a causa del timing, il 30% per la mira, il 20% per l'utility, il 10% per la posizione — quindi lavora sul timing questa settimana". È il consiglio di un tutor, non il giudizio di un giudice.

Confronta con: §4.3 (Il Commentatore del Valore — l'altra metà del giudizio: non "perché è andata male" ma "quanto è buono adesso"), §6.3 (L'Esame — le perdite che il detective minimizza).

§4.3 Il Commentatore del Valore

Il **Value Critic** del RAP Coach è un **commentatore sportivo in presa diretta** che ad ogni istante ti dice *"Ora hai il 72% di vantaggio"* o *"Il team è in svantaggio, 34% di chance di vincere il round"*. A differenza della probabilità di vittoria del **WinProbability** engine (che guarda solo al round, §4.4), il Value Critic guarda al **momento specifico** — la singola decisione — e stima quanto è buona dal punto di vista del coaching: *se continuo a comportarmi così, come va a finire il round?*. È il cuore della *pedagogia* del RAP Coach: senza una stima di valore, il modello non saprebbe se un consiglio sia rischioso o prudente, aggressivo o passivo. Il Value Critic si allena con MSE contro un target di ritorno temporalmente-scontato (classica RL value function) e in inferenza produce un numero tra -1 e +1 che si combina con l'attribuzione causale (§4.2) e con il vettore di advice probabilities per produrre il consiglio finale *"push con 0.72 di fiducia, perché value alto, timing dominante, economia stabile"*. Il commentatore non sostituisce il detective: lavorano insieme. Il detective dice *"hai sbagliato il timing"*; il commentatore dice *"e quel timing costerà il round"*. Insieme diventano una cronaca pedagogica: la diagnosi + la conseguenza. La differenza tra un commentatore con esperienza e uno alle prime armi è esattamente la differenza tra una value function ben calibrata e una rumorosa. Un commentatore esperto legge la partita

e sa dire *"adesso è un momento di svolta"* mentre uno novizio urla *"gol!"* ogni volta che la palla si avvicina alla porta. Il Value Critic bene addestrato distingue i momenti pivotali da quelli neutri; uno mal addestrato tira consigli a caso.

Confronta con: §4.2 (Il Detective — la metà "perché"), §4.4 (L'Albero di Scacchi — la stima di vittoria a scala round).

§4.4 L'Albero di Scacchi CS2

Per scenari strategici complessi — *"Eco-force CT su Inferno 1v3, bomba piantata, 18 secondi"* — il coach esplora un **albero di gioco expectiminimax** profondo tre livelli, come un motore di scacchi. A ogni nodo, uno dei giocatori (o il caso, per nodi probabilistici) ha una mossa; l'albero stima il valore atteso di ciascuna e sceglie la migliore. Accanto c'è uno **stimatore bayesiano di pericolo** (**BeliefModel**) che calcola *"quanto è probabile che io muoia ora?"* dati tre segnali: posizione (sono in una zona a letalità storica alta?), equipaggiamento (AWP vs pistola?) e stato dei nemici (li ho già sentiti? li ho già visti?). La formula è bayesiana classica: prior da pro stats, likelihood dai tick recenti, posterior da entrambe. Una calibrazione automatica tiene sotto controllo la bontà del prior per evitare overconfidence. Insieme, l'albero + lo stimatore forniscono una **valutazione strategica** che il coach comunica come *"sopravvivenza al 23%, push consigliato solo se hai informazioni sul secondo nemico"*. Tre livelli non sono profondi quanto un motore di scacchi da Top-10 FIDE, ma sono sufficienti per coprire *"duello diretto + trade + follow-up"*, che è il nucleo tattico di CS2. Più profondo diventerebbe inaffidabile, perché la variance del modello di transizione cresce esponenzialmente con il numero di tick simulati. La scelta di tre livelli è un compromesso calibrato empiricamente: a livello 1 l'albero non coglie le conseguenze; a livello 5 il rumore del modello di transizione (la difficoltà di predire cosa faranno i nemici tra 3 secondi) rende la valutazione peggiore di quella bayesiana stand-alone. A livello 3 si cattura il *"duello → trade → follow-up"* che è il pattern tattico dominante, e i rami vengono potati con alpha-beta pruning adattato al contesto probabilistico.

Confronta con: §4.5 (Il Poker-Face — un'altra misura probabilistica, sul bluff), §7.1 (La Squadra dei Detective — il Win Probability è uno dei detective).

§4.5 Il Poker-Face dell'Inganno

Il **DeceptionIndex** è un **punteggio poker-face**: quantifica quanto sei imprevedibile rispetto al giocatore professionista medio sulla stessa mappa nello stesso side. Conta flash fakes (hai simulato un apertura lanciando flash senza spingere?), fake sites (sei sparito dal radar verso B per poi tornare a A?), walk/run mixing (alterni movimento silenzioso e rumoroso per disorientare?), pre-utility tosses (lanci granate prima di muoverti per confondere il timing?). Ogni voce ha un peso tarato sulla frequenza con cui i pro lo fanno in quella situazione, e l'indice finale è una standardizzazione z-score: 0 = giochi esattamente come la mediana pro, +2 = sei chiaramente più ingannevole del pro medio, -2 = sei completamente leggibile. Un deception index basso non è automaticamente cattivo — ci sono ruoli (anchor pur, lurker disciplinato) dove la prevedibilità è funzionale — ma un indice cronicamente basso per un entry fragger è un campanello d'allarme: ti stanno prevedendo, e quella è la spiegazione più probabile dei tuoi duelli persi. Il

coach lo comunica con un grafico radar a 5 assi e con la voce *"oggi giochi in modo troppo lineare: prova a cambiare il lato della flash su Mirage mid quando il CT ti ha letto due volte di fila"*. L'Indice dell'Inganno è uno degli indicatori più sottili del coach: una volta interiorizzato, il giocatore riesce a gestire attivamente la propria prevedibilità — diventare più letto in alcuni momenti (per costruire pattern che poi romperà) e meno letto in altri (per massimizzare il vantaggio della rottura). È la mente strategica che emerge dal gameplay meccanico, e il coach fornisce la lente per vederla.

Confronta con: §7.2 (L'Anello dell'Umore — deception + momentum insieme raccontano la personalità del round), §4.2 (Il Detective — le 5 materie di cui deception è un sovra-livello).

§5 — Conoscenza e Recupero

§5.1 La Biblioteca FAISS

La base di conoscenza tattica è una **biblioteca** organizzata per concetti, non per parole chiave. Quando il coach vuole sapere *"come si gioca il retake CT sul sito B di Inferno"* non fa una ricerca full-text: chiede a **FAISS** (Facebook AI Similarity Search) il documento il cui embedding a 384 dimensioni è più vicino alla query in spazio coseno. I 384 numeri per documento sono prodotti da un modello Sentence-BERT che capisce semantica: sa che *"retake lato B"* e *"recupero del sito B"* sono la stessa cosa anche se le parole sono diverse. La biblioteca contiene il Coach Book v4: 502 voci tattiche in 8 file JSON (7 mappe attive + general) con 13 categorie — positioning, utility, economy, aim_and_duels, mid_round, retakes_post_plant, anti_strat, role_play (entry/awp/support/lurk/igl), clutch_play. Ogni voce è un paragrafo prescrittivo scritto per assomigliare a *"il tuo coach che ti spiega cosa fare adesso"*. La ricerca restituisce Top-K voci (tipicamente K=5) con un boost contestuale di 1.2× per voci che corrispondono esattamente alla mappa/side del giocatore e una deduplicazione a soglia 0.85 per evitare che cinque paragrafi quasi-identici appaiano nello stesso consiglio. È l'equivalente di un bibliotecario che non ti dà cinque copie dello stesso libro anche se sono tutte rilevanti. La scelta di 384 dimensioni non è casuale — è il footprint standard di Sentence-BERT, che offre il miglior compromesso tra precisione semantica e velocità di ricerca. Un embedding più grande catturerebbe sfumature più sottili ma rallenterebbe la ricerca; uno più piccolo sarebbe veloce ma perderebbe distinzioni semantiche cruciali. FAISS è configurato in modalità "flat" (ricerca esaustiva) perché 502 voci non richiedono strutture ANN (Approximate Nearest Neighbor) più complesse; per knowledge base molto più grandi (migliaia di voci) si passerebbe a HNSW o IVF, ma a quel punto si pagherebbe un piccolo errore di ricerca in cambio della velocità.

Confronta con: §5.2 (Il Diario — la conoscenza esperienziale che integra la biblioteca), §5.3 (Il Grafo — le connessioni tra voci).

§5.2 Il Libro di Testo + Diario

Il coach ha **due fonti di conoscenza** che usa in parallelo, come uno studente che studia per un esame. Il **libro di testo** è il RAG (§5.1): conoscenza pubblica, consolidata, scritta da coach umani. Il **diario personale** è il COPER [ExperienceBank](#): ogni volta che il coach dà un consiglio durante una partita, scrive una voce nel diario — *"Dust2, round eco side T, giocatore nei tunnel B con 60 HP e una Deagle. Gli ho detto di tenere l'angolo. È sopravvissuto e ha preso 2 uccisioni. Il consiglio ha FUNZIONATO"*. Più tardi, quando si presenta una situazione simile, il coach consulta sia il libro di testo (cosa dicono i coach in generale?) sia il diario (cos'è successo l'ultima volta che questo giocatore specifico era in questa situazione?). Il risultato è un consiglio ibrido che parla con autorità e con memoria personale. Il KT-01 porta il diario al livello successivo con **semantica CRUD** (UPDATE/DISCARD/KEEP), **replay prioritizzato** (situazioni ad alto impatto vengono rivisitate più spesso) e **integrazione TrueSkill** (il peso di ogni voce cresce con l'accuratezza storica del coaching). Un ciclo di feedback EMA ($new_score = 0.7 \times old + 0.3 \times outcome$) fa sì che buoni consigli diventino più affidabili, quelli cattivi si attenuino, e dopo 90 giorni senza convalida la fiducia in una voce decade del 10%: come una previsione meteorologica che perde affidabilità man mano che ci si allontana dal momento in cui è stata fatta. La combinazione libro-di-testo + diario è potente perché i due sistemi coprono spazi di conoscenza complementari. Il libro di testo è generale, impersonale, stabile — ti dice *cosa i coach del mondo hanno imparato*. Il diario è specifico, personale, in evoluzione — ti dice *cosa ha funzionato per questo giocatore*. Un coach che avesse solo il libro di testo darebbe consigli generici; uno che avesse solo il diario sarebbe cieco davanti a situazioni nuove. Insieme, diventa un tutor che conosce sia la teoria che te.

Confronta con: §4.1 (L'Ospedale — la reception che integra libro + diario), §5.1 (La Biblioteca — il libro di testo), §2.3 (Le Lenti — cosa finisce nel diario).

§5.3 Il Grafo dei Fatti

Oltre a FAISS e all'ExperienceBank, esiste un **grafo di conoscenza** che collega i fatti del Coach Book v4 tra loro via relazioni semantiche — *"questa voce su Mirage mid è il contro-strat di quella voce sul rush A con flash over"*, *"questa tattica pro è l'evoluzione di una tecnica più antica"*. Il grafo abilita **reasoning multi-hop**: a fronte di una domanda specifica, il coach può esplorare un percorso — concetto A → relazione → concetto B → relazione → concetto C — per costruire una risposta composta piuttosto che restituire un singolo paragrafo. È come un esperto di scacchi che spiega una mossa riferendosi non solo a quella mossa, ma al sistema di aperture a cui appartiene, al piano strategico tipico, al finale a cui conduce. Il grafo è denso nei punti dove il gameplay di CS2 è più ricco (default sul mid di Mirage, retake sul sito B di Inferno, eco management) e più rado nelle aree marginali (meta secondarie, strat obsoleti). Nell'implementazione il grafo è in memoria — non persistito in un graph database dedicato — e viene ricostruito da un orchestratore all'avvio dei servizi di coaching. È una struttura viva: man mano che l'ExperienceBank (§5.2) accumula voci, nuovi edge vengono proposti automaticamente per integrare l'esperienza specifica del giocatore con la conoscenza pubblica. Il multi-hop è ciò che distingue un coach maturo da un coach giovane: un coach giovane conosce singole giocate; un coach maturo vede *strutture*

di gioco. Il grafo codifica esattamente queste strutture, e il coach impara man mano a navigarle — all'inizio salta da un nodo all'altro in modo sordo, dopo molti demo segue pattern coerenti che fanno emergere consigli di livello strategico alto.

Confronta con: §5.1 (La Biblioteca — il grafo navigato da FAISS), §5.2 (Il Diario — come si aggiungono edge al grafo).

§6 — Addestramento e Orchestrazione

§6.1 La Scuola del Coach

L'addestramento del coach è una **scuola con un curriculum di 4 fasi**. Fase 1 è l'**asilo** (CALIBRATING, 0-49 demo ingerite): il coach guarda partite come un bambino guarda documentari di cucina — assorbe pattern senza cucinare. Fase 2 è la **scuola media** (LEARNING, 50-199 demo): passa dai documentari alla scuola di cucina, segue ricette semplici sotto supervisione. Fase 3 è il **liceo professionale** (MATURING, transizione): cucina per amici, riceve feedback, impara dai propri errori. Fase 4 è il **diploma** (MATURE, 200+ demo): diventa master chef capace di inventare piatti nuovi. Il `CoachTrainingManager` è il **preside** della scuola: decide quale modello può studiare cosa e quando, tiene traccia dei voti tramite l'Osservatorio (§1.7), e scala il moltiplicatore di fiducia dei consigli in base alla fase corrente ($0.5\times \rightarrow 0.8\times \rightarrow 1.0\times$). Ogni modello ha il proprio curriculum: JEPA studia pre-training auto-supervisionato con split context/target e InfoNCE; VL-JEPA impara i 16 concetti di coaching come sedici materie scolastiche (positioning, utility, economy, engagement, decisione, psicologia); RAP Coach impara la *pedagogia* con una perdita composita. Il **daemon Teacher** aggiorna il curriculum quando la biblioteca cresce del 10% — il preside non fa ripetere l'anno se non ci sono abbastanza nuovi esempi, perché ripetere senza novità significa overfitting, e overfitting significa che il coach impara a memoria le partite invece di impararne i principi. La scala delle 4 fasi è calibrata empiricamente: un modello con 30 demo ha visto troppo poca varietà per giudicare con confidenza un giocatore nuovo; a 100 demo inizia a riconoscere i pattern tipici; a 200 demo la sua calibrazione interna è stabile. Spingere il coach in produzione troppo presto (sotto i 50 demo) produce raccomandazioni "scatola nera" che possono confondere invece di aiutare; aspettare troppo a lungo (oltre i 400 demo) porta a rendimenti decrescenti. La scala è una scelta ingegneristica di prudenza.

Confronta con: §1.7 (La Pagella — come si misura il progresso scolastico), §6.2 (L'Orchestra — chi esegue il curriculum).

§6.2 L'Orchestra del Training

Il `TrainingOrchestrator` è il **direttore d'orchestra** che porta i modelli attraverso epoche, validazione, early stopping e checkpoint. Ogni modello è uno strumento: JEPA è il primo violino (pre-training rappresentazionale, `JEPATrainer`), VL-JEPA è il secondo violino (allineamento concettuale), RAP Coach è l'orchestra da camera (perdita composita multi-task, `RAPTrainer`), NeuralRoleHead è il percussionista

(classificazione rapida). Il direttore non suona strumenti — dirige: stabilisce il tempo (learning rate schedule), segnala i cambi di movimento (early stopping), e quando serve richiama un musicista per un assolo (focal training su una metrica specifica). I **callback** sono come **sensori di un'auto da corsa**: raccolgono dati senza interferire con la guida. TensorBoard callback registra loss, accuracy, gradient norms, attention patterns; il ConvictionCallback aggiorna i cinque segnali dell'Osservatorio; il DriftMonitor osserva z-score delle distribuzioni di feature e alza un allarme quando la deriva supera 2.5 deviazioni standard — *"il meta è cambiato, è ora di retrain"*. Nei callback vige una regola ferrea: **non interferire con il training**. Se un callback si rompe, il training deve continuare; se un callback è lento, non può rallentare le epoche. Ogni callback è un osservatore indipendente, come le 50 telecamere che riprendono un concerto ma nessuna può fermare la musica. La ragione per cui i callback sono sacri è che sono la fonte della auto-diagnostica: senza di loro, un training che diverge viene scoperto solo dopo ore di GPU sprecate. Con loro, il training si ferma alla prima irregolarità significativa. Il prezzo di questa sicurezza è l'overhead di calcolo — ogni callback ruba una frazione di tempo — ma il guadagno è impagabile.

Confronta con: §1.7 (La Pagella — come leggere i dati dei sensori), §6.3 (L'Esame — le metriche specifiche che l'orchestra esegue).

§6.3 L'Esame con Risposte sul Retro

Le **funzioni di perdita** dell'addestramento sono come **compiti d'esame**, e ogni tipo di esame misura una cosa diversa. **MSE** (Mean Squared Error) è un problema di matematica numerico: "qual è il valore esatto?". **Cross-Entropy** è un quiz a risposta multipla: "qual è la categoria corretta tra K?". **BCE** (Binary Cross-Entropy) è una serie di domande sì/no. **InfoNCE** è un test di identificazione in un gruppo: "tra 32 persone, chi è la corrispondenza giusta per questa foto?" — il τ (temperature) regola quanto sono *nitide* le lenti del riconoscimento (τ piccolo = lenti precise ma fragili; τ grande = lenti sfocate ma tolleranti). **KL Divergence** misura quanto due distribuzioni di probabilità assomigliano a quella target. **Sparsity L1** è il vincolo "usa il minor numero possibile di neuroni" (§1.3). **VICReg** mantiene la classe variegata: senza VICReg, tutti gli studenti danno la stessa risposta; con VICReg, le risposte devono essere diverse (varianza alta) e non correlate (covarianza bassa). Il peccato capitale dell'addestramento è la **label leakage** — un esame con le risposte stampate sul retro del foglio: se nei tuoi 25 feature c'è `round_won` (invariante `P-RSB-03`), il modello impara a leggere il retro del foglio invece di studiare, e in produzione, quando il retro non c'è, non sa più rispondere. Per questo `round_won` è proibito nei feature di training. La caccia alla label leakage è una disciplina quasi religiosa: ogni volta che si introduce un nuovo feature, bisogna chiedersi *"può essere calcolato solo dopo la fine del round?"*. Se la risposta è sì, è leakage. Esempi subdoli: `final_kda` (disponibile solo a fine round), `round_money_earned` (include il bonus vittoria), `time_since_last_defuse` (solo se il bomb è stato piantato, cioè il round è quasi finito). Ogni feature passa un audit prima di entrare nel vettore a 25 dimensioni.

Confronta con: §6.2 (L'Orchestra — chi somministra gli esami), §1.7 (La Pagella — dove vengono scritti i voti).

§6.4 Il Talent Scout dei Ruoli

Il **RoleClassifier** è un **talent scout** che guarda un giocatore per 10 round e dice "questo è un AWP", "questo è un entry", "questo è un anchor", "questo è un support", "questo è un lurker". Ma il talent scout rispetta una regola: **non decide finché non ha visto abbastanza**. Il **RoleThresholdStore** è come un medico cauto che rifiuta di diagnosticare una patologia rara se ha visto solo due pazienti — ha bisogno di un campione minimo (tipicamente 30 round di osservazione) prima di assegnare un ruolo con confidenza > 0.7 . Finché la soglia non è raggiunta, il ruolo resta UNKNOWN e il coaching si adatta: consigli generici invece di consigli ruolo-specifici. Quando il campione cresce, entra in gioco il **NeuralRoleHead**: un MLP a 5 uscite che fa un **quiz lampo** basato sui 25 feature aggregati del giocatore. Il quiz e il talent scout classico lavorano in consenso: se concordano, la classificazione è accettata; se divergono significativamente, il coach segnala "ruolo ambiguo, il giocatore sta esplorando". La **cold-start protection** serve perché un maestro di scuola che non conosce ancora gli studenti non deve etichettarli al primo giorno — e nemmeno il coach. La dualità scout+quiz è saggia: lo scout classico è basato su regole euristiche scritte da umani ("ha usato l'AWP nel 70% dei round? è un AWP"); il quiz neurale è basato su pattern appresi dai dati. Le regole euristiche sono trasparenti ma rigide; la rete neurale è flessibile ma opaca. Usandoli insieme, il coach ha un sistema che è trasparente dove le regole convergono ("entrambi dicono AWP") e attento all'incertezza dove divergono ("lo scout dice entry, la rete dice support — esploriamo").

Confronta con: §4.5 (Il Poker-Face — un'altra misura di "chi sei" ma sul piano dell'imprevedibilità), §4.2 (Il Detective — i ruoli sono il contesto dell'attribuzione).

§7 — Motori di Analisi

§7.1 La Squadra dei Detective

L' **AnalysisOrchestrator** coordina una **squadra di undici detective specializzati**, ognuno con il proprio libretto di tecniche investigative. Il Detective Statistico calcola HLTV 2.0 rating (vedi §9 del Book 2: kills, deaths, ADR, KAST%, impact, survival, flash assists). Il Detective del Momentum segue le tendenze round-per-round (§7.2). Il Detective del Tempo di Ingaggio studia a quale distanza il giocatore ingaggia più spesso (close/medium/long) e lo confronta con il baseline pro per il suo ruolo. Il Detective delle Utility controlla se le granate sono spese efficacemente (§7.3 adiacente). Il Detective della Probabilità di Vittoria stima round-by-round con una rete integrata Elo (§4.4 vicino). Il Detective dell'Entropia misura la sorpresa delle decisioni. Il Detective dell'Inganno (§4.5) e dell'Economia completano il team. Undici detective condividono un codice: **ogni indagine è autonoma**, nessuno chiede all'altro dati intermedi, tutti leggono dal **MatchDatabase** e scrivono in tabelle separate. Questo disaccoppiamento è cruciale: se un motore si rompe, gli altri continuano; se un motore migliora, gli altri non devono essere ri-testati. L'orchestratore raccoglie le conclusioni di tutti e le consegna al **CoachingService** (§4.1), che decide quale insight portare in primo piano in base al contesto del consiglio. La scelta di undici detective anziché di un singolo "super-motore" è architettónica. Un singolo motore che fa tutto sarebbe un enorme modello monolitico difficile da

mantenere, debuggare, sostituire. Undici motori specializzati sono come undici esperti di una specializzazione ciascuno: ognuno può essere aggiornato senza toccare gli altri, ognuno può essere testato in isolamento, ognuno può raccontare la sua storia in modo comprensibile. È la stessa ragione per cui gli ospedali hanno reparti separati anziché un unico "specialista dell'anima umana".

Confronta con: §4.4 (L'Albero di Scacchi — un detective specifico per la probabilità strategica), §7.2 (L'Anello dell'Umore — un detective specifico per il momentum).

§7.2 L'Anello dell'Umore

Il **MomentumTracker** è come un **anello dell'umore** che cambia colore in base all'andamento della partita. Ogni round ha un *momentum score* calcolato da segnali combinati: tempo rimanente al momento della chiusura, numero di kill consecutivi senza perdite, sopravvissuti del round precedente, equipment differential. Il tracker tiene una finestra mobile di cinque round e produce stati discreti — ICE (momentum congelato, spirale negativa), COOL (controllo), WARM (buon ritmo), HOT (streak positiva), VOLCANIC (round-vincente da highlight). L'anello si raffredda con una decay costante: se non vinci round per un po', torni naturalmente verso COOL indipendentemente da quanto eri HOT prima. Integrata all'anello c'è una **mappa del calore** (heatmap) che visualizza dove i giocatori pro tendono a trovarsi nella stessa situazione — se la tua heatmap differisce dal baseline pro in modo significativo, il coach segnala *"posizionamento non standard, ecco il riferimento dei pro"*. La heatmap usa 128×128 celle per ogni mappa CS2 con calibrazione pixel-perfect (vedi la sezione sui mapping nel README root). Insieme, momentum + heatmap raccontano la *temperatura emotiva* del round e la geografia delle posizioni vincenti, due pezzi che il coach combina in feedback come *"sei HOT ma stai giocando su posizioni fuori meta — consolida prima di spingere"*. Il momentum è un segnale che i giocatori umani sentono intuitivamente ma non quantificano. L'anello dell'umore rende visibile l'intuizione: in un grafico, il giocatore vede esattamente il momento in cui il suo ritmo è cambiato, e può correlarlo con una scelta specifica. Questa visibilità è formativa: aiuta a sviluppare consapevolezza emotiva, una meta-capacità che si traduce poi in decisioni migliori.

Confronta con: §2.3 (Le Tre Lenti — il momentum è una lente macro), §4.5 (Il Poker-Face — altra misura di come suoni al nemico).

§7.3 L'Istruttore di Guida

Il **BlindSpotDetector** e il **MovementQualityAnalyzer** formano l'**istruttore di guida del coach**.

L'istruttore di guida nota quando dimentichi di controllare uno specchietto, quando acceleri su una strada bagnata, quando freni tardi in curva. Allo stesso modo, il **BlindSpotDetector** registra gli angoli che il giocatore ha consistentemente omesso di controllare durante i 10 round precedenti su una certa mappa — *"su Mirage mid, controlla sempre window ma dimentichi sempre triple box"* — e alza un alert visivo sulla mappa tattica con un cerchio giallo nell'angolo cieco. Il **MovementQualityAnalyzer** è l'istruttore che guarda i replay in slow-motion: strafe jumping pulito o grezzo? counter-strafting accurato prima di sparare o scivoli mentre premi? peek shoulder-first o chest-first? Ogni tecnica ha una metrica da 0 a 100 basata sulla differenza rispetto al movimento standard dei pro, e l'analyzer restituisce un report tecnico con

priorità di miglioramento. L'istruttore non urla mai — rileva il pattern e lo comunica con calma: *"nelle ultime 20 partite, 12 morti sono avvenute durante peek con strafe non finalizzato. Ti consiglio 15 minuti di counter-strafe drill prima della prossima partita"*. La pedagogia non è rabbia — è diagnosi precisa seguita da un esercizio mirato. Il movimento in CS2 è uno dei livelli più sottili del gioco — un giocatore può avere mira eccellente e decisioni strategiche perfette ma perdere duelli per micro-difetti di movimento che non percepisce consciamente. L'istruttore di guida rende visibili questi micro-difetti e li trasforma in obiettivi di training concreti. Dopo qualche settimana di drill mirato, il giocatore sente letteralmente la differenza: i duelli che prima perdeva per 50 ms di ritardo li vince; il counter-strafe diventa automatico; la peek si sincronizza con il crosshair.

Confronta con: §7.1 (La Squadra — l'istruttore è uno degli undici detective), §4.2 (Il Detective dei 5 Errori — le cinque categorie che l'istruttore diagnostica).

§8 — Dati e Ingestione

§8.1 L'Ufficio Postale della Lettera

La pipeline di ingestione è un **ufficio postale** in cui ogni demo CS2 è una lettera che attraversa sette tappe: **(1) Scoperta** — il daemon Scanner trova il file `.dem` in una cartella configurata; **(2) Validazione** — l' `IntegrityChecker` verifica che il file superi `MIN_DEMO_SIZE` (10 MB, invariante DS-12) e che sia una demo CS2 valida; **(3) Registrazione** — il demo viene iscritto nel `DemoRegistry` con hash MD5 per deduplicazione; **(4) Parsing** — il `DemoParser` estrae ogni tick via `demoparser2` in un database SQLite per-match; **(5) Feature Extraction** — il `FeatureExtractor` applica il contratto a 25 dimensioni (§3.2); **(6) Stat Calculation** — l' `Analyzer` calcola HLTV 2.0, round stats, economia; **(7) Training Queue** — se la soglia del 10% di crescita è raggiunta, il daemon Teacher viene notificato. L'*ufficio postale* ha un supervisor — `run_ingestion.py` — che coordina le sette tappe, gestisce gli errori (demo corrotti, file troppo piccoli, duplicati) e produce statistiche giornaliere. Ogni stadio ha una regola: **non si può saltare**. Una lettera senza validazione è una lettera rischiosa; una lettera senza registrazione è una lettera che può essere processata due volte; un parsing senza feature extraction è sforzo sprecato. Il `run_worker.py` è il **postino di supporto**: recupera task stale (> 5 minuti in coda) da un altro processo che si è interrotto, evitando che una demo finisca a metà per sempre. La sequenzialità delle 7 tappe è una scelta di robustezza: ogni stadio è un checkpoint, e se il sistema crasha tra due stadi, la demo si riprende dall'ultimo completato invece di ricominciare da capo. Questo è critico perché il parsing di un demo (stadio 4) può impiegare diversi minuti per i demo più lunghi — ricominciare da zero dopo un crash produrrebbe cicli di fallimento infiniti.

Confronta con: §8.2 (Il Doganiere — il controllo qualità della tappa 2), §6.1 (La Scuola — la tappa 7 vista dalla scuola).

§8.2 Il Doganiere Demo

Il `DemoFormatAdapter` è il **doganiere** che controlla i pacchi in ingresso. Ogni demo CS2 che arriva all'ufficio postale (§8.1) viene ispezionata: dimensione > 10 MB (`MIN_DEMO_SIZE` , `demo_format_adapter.py:49` , invariante DS-12 — un demo pro legittimo è tipicamente 300-850 MB, sotto 10 MB significa sicuramente corrotto o incompleto), versione del protocollo corretta, signature CS2 presente. Il `DemoQualityScorer` è l'**ispettore di qualità** che fa un controllo più profondo dopo il doganiere: il demo ha 15 round o è una partita Short? Il nome dei player è leggibile? Il server tick rate è 128? Il map è nel pool Active Duty (7 mappe: anubis, ancient, dust2, inferno, mirage, nuke, overpass)? In base a queste ispezioni, ogni demo riceve un *quality score* tra 0 e 1. Demo con score < 0.6 vengono flaggati per revisione manuale; demo con score < 0.3 vengono rifiutati. Il doganiere è spietato perché una singola demo corrotta può inquinare il training: se includi un demo con tick rate sbagliato o un demo corrotto a metà, il modello impara pattern falsi. La lezione del doganiere è severa: meglio 99 demo eccellenti che 100 demo di cui una rovinata. Questo principio è controintuitivo — "*più dati = meglio*" è un mantra del ML, no? Ma nel contesto del coaching il principio non si applica: un modello addestrato su demo di bassa qualità produce consigli di bassa qualità, indipendentemente dalla quantità. Il doganiere è la prima linea di difesa contro la degenerazione silenziosa della qualità, e il coach investe in questa rigore perché ne raccoglie i benefici a valle, in forma di consigli più accurati.

Confronta con: §8.1 (L'Ufficio Postale — dove vive il doganiere), §9.2 (La Cassaforte — dove finisce la demo dopo il controllo).

§8.3 Il Cronista Sportivo HLTV

Il `DemoParser` è un **cronista sportivo esperto** che, guardando un demo, compila un report card dettagliato tick-by-tick: chi ha ucciso chi, con quale arma, a quale distanza, dopo aver assassinato la flash lanciata da chi, quanti secondi prima che il bomba venisse piantata. Il cronista non inventa — riporta. Non interpreta — registra. L'interpretazione arriva dai motori di analisi (§7.1). Il modulo HLTV (`hlvtv_sync_service.py` , `fetch_hlvtv_stats.py` se presente) è la **squadra di spionaggio** — ben organizzata, discreta, rispettosa delle regole — che raccoglie statistiche dei pro dai siti HLTV. Usa Playwright per il browser headless, FlareSolvrr Docker per bypassare Cloudflare, un `RateLimiter` per rispettare i limiti del sito, un `CircuitBreaker` (§9.2 adjacente) che apre il circuito se HLTV risponde con errori consecutivi e ri-tenta con backoff esponenziale. Il risultato è il database `hlvtv_metadata.db` con tre tabelle: `ProPlayer` , `ProTeam` , `ProPlayerStatCard` (161 giocatori pro, 32 team, 156 stat card al momento della scrittura). Il sync service vive in un processo separato dai daemon del session engine, per evitare contese WAL sul database monolite. Il rispetto delle regole del sito HLTV è deliberato — il coach non si comporta come un bot aggressivo, non fa scraping intenso, non tenta di bypassare sistemi di protezione oltre a FlareSolvrr (necessario per aggirare Cloudflare legittimamente). La filosofia è di essere un buon vicino della comunità CS2: HLTV fornisce dati per la comunità, e il coach ne usa una porzione limitata per il confronto con i pro, senza stressarli.

Confronta con: §3.1 (I 5 Sensi — il sesto senso del coach), §9.1 (L'Archivio — dove il cronista archivia i suoi reporti).

§9 — Database e Storage

§9.1 L'Archivio a 3 Livelli

Il sistema di storage è un **archivio a tre livelli**, ciascuno progettato per evitare un tipo diverso di problema. Il **livello 1** è `database.db`, il monolite principale (18 tabelle SQLAlchemy definite in `database.py:_MONOLITH_TABLES` righe 54-73): statistiche giocatori, stato del coach, task di ingestione, insight di coaching, RAG knowledge, COPER experience bank, calibrazioni. Il **livello 2** è `hlvtv_metadata.db` (3 tabelle: `ProPlayer`, `ProTeam`, `ProPlayerStatCard`), separato dal monolite perché viene scritto da un processo esterno (HLTV sync service) e la separazione elimina la contesa WAL tra quel processo e i daemon del session engine. Il **livello 3** è `match_data/{id}.db`: un database SQLite dedicato per ogni partita, contenente i dati tick-per-tick (~100.000 righe per partita) — tre tabelle in ogni DB: `MatchTickState:110`, `MatchEventState:190`, `MatchMetadata:242` in `match_data_manager.py`. Questa separazione risolve il *Telemetry Cliff*: impedisce che il monolite cresca indefinitamente con telemetria ad alta frequenza. La **modalità WAL** (Write-Ahead Logging) è la porta girevole dell'archivio: lettori e scrittori concorrenti entrano senza urtarsi, perché la scrittura avviene in un file separato (.wal) che viene periodicamente consolidato. Totale: 24 tabelle SQLAlchemy distribuite su 3 tier, con 5 PRAGMA (`database.py:110-118`): WAL on, synchronous NORMAL, foreign_keys ON (DB-06), wal_autocheckpoint 512 (DB-07), busy_timeout 5000. L'architettura a 3 livelli non è solo ingegneria — è una filosofia dei dati. Dati diversi hanno esigenze diverse: il monolite ha dati caldi a bassa frequenza (player stats che cambiano raramente), l'HLTV ha dati esterni aggiornati settimanalmente, la telemetria per-match ha dati immensi e "archiviabili" dopo l'analisi. Metterli insieme creerebbe contese, rallentamenti, e farebbe esplodere il file del monolite. Separarli permette a ciascun livello di ottimizzare indipendentemente.

Confronta con: §8.1 (L'Ufficio Postale — la tappa 3 manda le demo qui), §9.2 (La Cassaforte — chi protegge l'archivio).

§9.2 La Cassaforte del Backup

Il `BackupManager` è la **cassaforte di una banca** — un sistema automatico che copia ogni database in un posto sicuro a cadenza stabilita senza mai interrompere l'archivio principale. Usa la SQLite **Online Backup API** (`sqlite3.Connection.backup()`, `backup_manager.py:81-89`, invariante BE-01/DB-03) invece di `VACUUM INTO`, perché l'Online Backup non richiede lock esclusivi e non blocca i writer live — è come fotocopiare un libro senza rimuoverlo dallo scaffale. La politica di retention è "1 latest + 7 daily + 4 weekly": il backup più recente viene sempre conservato, poi gli ultimi 7 backup giornalieri e gli ultimi 4 weekly (settimanali consolidati). Backups più vecchi del weekly più anziano vengono eliminati, per evitare che la cassaforte esploda. Accanto al BackupManager, il sistema di **tolleranza ai guasti** è una sicurezza nucleare a più strati: (1) pre-commit hooks bloccano codice che viola invarianti; (2) il daemon Pulse riavvia un daemon che si è fermato; (3) il `CircuitBreaker` degli scraper HLTV evita di martellare un sito che risponde con errori (stati: CLOSED/OPEN/HALF_OPEN); (4) lock di singleton a file system impediscono che due istanze dell'app si scrivano addosso. Il principio è lo stesso delle centrali nucleari:

uno strato può fallire, due strati possono fallire, ma perché il sistema ceda devono fallire tutti contemporaneamente. La cassaforte ha un'altra funzione oltre al ripristino: è una **time machine** per il debugging. Quando si sospetta che uno stato sia stato corrotto — per esempio una stat card pro sembra sbagliata — si può aprire un backup di 3 giorni prima e vedere se il valore corrotto era già lì. Questo permette di tracciare bug subdoli che altrimenti sarebbero invisibili.

Confronta con: §9.1 (L'Archivio — ciò che la cassaforte protegge), §6.2 (L'Orchestra — un altro sistema a strati).

§10 — Visione d'Insieme

§10.1 Il Robot che Guarda Video

Immagina di avere un **allenatore robot super intelligente** che guarda le tue partite di CS2 come videoregistrazioni. Innanzitutto, **guarda** centinaia di partite professionistiche e le tue, prendendo appunti tick per tick (questa è l'ingestione, gestita dal daemon Scanner e dal Digester). Poi, **studia** gli appunti e impara cosa fanno i grandi giocatori in modo diverso dai principianti, come uno studente che avanza nella scuola (CALIBRATING = asilo, LEARNING = media, MATURE = diploma) — questo lo fa il daemon Teacher in background. Quando è il momento di darti un consiglio, non tira a indovinare: consulta il suo **quaderno di suggerimenti** (RAG), la **memoria delle sessioni passate** (COPER) e cosa farebbero i **pro** nella tua esatta situazione, scegliendo la fonte di cui si fida di più in base alla maturità dei modelli. Infine, **spiega** perché: non solo *"fai questo"*, ma *"fai questo perché continui a essere colto alla sprovvista da mid control su Mirage"*. È come avere un coach che ha visto ogni partita pro mai giocata, ricorda ogni sessione di allenamento che hai fatto, e può spiegarti esattamente perché dovresti cambiare strategia. Nel frattempo, l'interfaccia Qt/PySide6 ti mostra tutto in tempo reale: mappa tattica 2D con il tuo ghost ottimale, grafici radar delle tue abilità, e una dashboard che ti dice a che punto è il tuo coach nel suo processo di apprendimento. Il daemon Pulse assicura che il sistema sia sempre vigile. Questa descrizione è la sintesi alta di tutto il sistema — la metafora da cui tutte le altre discendono. È il robot che guarda video che, messo in funzione, rivela ogni reparto, ogni modello, ogni motore di analisi. Gli altri tre capitoli di questa sezione scompongono il robot in viste diverse — infrastruttura urbana, architettura a strati — ma la metafora fondativa resta questa: un allenatore silenzioso, instancabile, che guarda, studia, ricorda, e spiega.

Confronta con: §10.2 (La Città — la stessa panoramica dal punto di vista dell'infrastruttura), §1.1 (La Fabbrica — la stessa panoramica dal punto di vista organizzativo).

§10.2 La Città del Coach

Dal punto di vista dell'infrastruttura, l'intero sistema è una **piccola città** autosufficiente. Il **municipio** è il `SessionEngine` (`core/session_engine.py`) che coordina i quattro daemon cittadini — Scanner, Digester, Teacher, Pulse — come quattro assessori con responsabilità distinte. La **centrale elettrica**

(session engine al livello più basso) ha quattro reattori che producono energia per tutti gli altri servizi; se uno va offline, gli altri tre continuano e Pulse invia un allarme. L'**archivio comunale** (storage, §9.1) è a tre livelli sotto il municipio. La **biblioteca pubblica** (knowledge, §5.1) e il **diario segreto del sindaco** (COPER, §5.2) sono nella stessa ala ma con permessi diversi. La **scuola** (training, §6.1) forma i cittadini adulti, i modelli. L'**ospedale** (coaching services, §4.1) cura i cittadini che chiedono consiglio. La **stazione di polizia** (RASP guard, [observability/rasp.py](#)) verifica l'integrità degli edifici attraverso hash SHA-256. L'**ufficio postale** (ingestione, §8.1) distribuisce i demo in arrivo. Il **cronista sportivo** (HLTV sync, §8.3) vive appena fuori città vicino allo stadio per ricevere le ultime notizie. Ogni edificio ha una porta — un endpoint API o un metodo pubblico — e nessun edificio bussa alle porte laterali degli altri: tutto passa attraverso la piazza (servizi centrali). La città è piccola ma completa: può funzionare senza internet, senza HLTV, senza Ollama — perdendo funzioni ma non crollando. La metafora della città evidenzia una qualità critica del sistema: la **sussistenza**. Il coach è stato progettato per funzionare offline, su hardware modesto, senza dipendenze cloud. Questa scelta non è solo tecnica ma filosofica — il coach deve essere del giocatore, non dei server di qualcun altro. Un giocatore deve poter eseguire il coach sul proprio PC senza chiedere permesso a nessuno, senza pagare abbonamenti, senza che nessuno guardi i suoi dati. La città del coach è una città sovrana.

Confronta con: §10.1 (Il Robot — la città vista dall'utente), §10.3 (Il Cruscotto — la plancia di controllo della città).

§10.3 Il Cruscotto a Cinque Strati

L'interfaccia desktop Qt/PySide6 è il **cruscotto di un'auto sportiva moderna**: tutto ciò che il guidatore (il giocatore) deve sapere è visibile a colpo d'occhio, ogni quadrante ha un significato preciso, e sotto il cruscotto c'è un **CAN bus** di segnali e slot — il pattern Signal/Slot di Qt — che fa dialogare 15 schermate con 7 ViewModels senza accoppiamento stretto. L'architettura complessiva è una **torta a cinque strati**: (1) UI Qt in cima ([apps/qt_app/screens/](#)), (2) ViewModels MVVM che espongono property osservabili, (3) Servizi di coaching e analisi (backend/services + backend/analysis), (4) Modelli neurali e logica pura (backend/nn + backend/processing + backend/coaching), (5) Storage (backend/storage) in fondo. I confini tra strati sono attraversati solo in una direzione — UI chiama ViewModel, ViewModel chiama Service, Service chiama Model o Storage — e mai viceversa, per evitare dipendenze circolari che trasformerebbero la torta in un plastico di spaghetti. La **cassetta degli attrezzi** del progetto contiene PyTorch (reti neurali), ncps (LTC), hopfield-layers (Hopfield), demoparser2 (demo parsing Rust-based), PySide6 (UI primaria), SQLAlchemy+SQLModel (ORM), Alembic (migrations), Playwright (HLTV scraping), Rich (console TUI), FastAPI (internal API), Pydantic (validazione). Ogni attrezzo è un artigiano specializzato: usi quello giusto per ogni lavoro, mai un martello per una vite. La torta a cinque strati è una delle invarianti architetturali più preziose del progetto — è ciò che rende il codebase manutenibile nonostante abbia superato le 100.000 righe di Python. Violare la direzionalità dei flussi (per esempio, far chiamare al modello un ViewModel) produrrebbe un code smell noto come *callback hell* o *architectural spaghetti*, e la manutenzione diventerebbe un incubo. La disciplina dei cinque strati, applicata con rigore, è il motivo per cui il coach può essere esteso nel tempo senza collassare sotto il proprio peso.

Confronta con: §10.2 (La Città — l'infrastruttura che il cruscotto comanda), §1.1 (La Fabbrica — i reparti che la torta a 5 strati codifica).

Nota finale dell'autore

Queste 35 analogie non esauriscono la ricchezza dei quattro libri-coach — ne sono la sintesi pedagogica. Dove un libro tecnico scriverebbe *"il SuperpositionLayer applica un gating contestuale attraverso L1-sparsity regolata"*, qui scriviamo *"il mixer a 256 canali abbassa automaticamente i volumi inutili"*. Nessuna delle due formulazioni è più corretta dell'altra — sono complementari. La prima è indispensabile per chi scrive il codice; la seconda è indispensabile per chi ne vuole capire l'intenzione.

Se durante la lettura di un capitolo ti trovi a pensare *"ok, tecnicamente chiaro, ma perché questa scelta?"*, torna a questo volume. Se ti trovi a pensare *"ok, metafora chiara, ma come si implementa?"*, torna al capitolo tecnico. Le due letture fatte insieme sono la comprensione piena.

Un ultimo pensiero. Scrivere analogie buone è più difficile che scrivere documentazione tecnica. Un documento tecnico ha un unico criterio di correttezza (corrisponde al codice, sì o no?); un'analogia ne ha molti: rispetta la pedagogia? è memorabile? è corretta nel punto in cui conta? resiste a critiche dei lettori tecnici? non crea false aspettative? Ogni voce qui è stata iterata più volte per raggiungere questo equilibrio. Se trovi che una metafora non regge — che crea un modello mentale sbagliato rispetto al codice — è un bug, ed è il tipo di bug più importante da segnalare, perché la pedagogia distorta è peggiore del silenzio.

Fine del Libro delle Analogie (versione canonica italiana). Le traduzioni inglese e portoghese seguono in [analogy-book-en.md](#) e [analogy-book-pt.md](#), parity-checked rispetto a questa versione.